

FirstKnock Revised Pipeline Architecture Spec

FirstKnock Revised Pipeline Architecture Spec

Production Developer Specification — Zero-Cost Classification Pipeline

April 7, 2026

Document Type	Production Developer Specification
Prepared By	Senior Backend Engineer — Pipeline Audit
Date	April 7, 2026
Status	Ready for Implementation
Supersedes	All prior pipeline architecture documents

Overview

This document specifies the revised FirstKnock data pipeline architecture. It replaces all prior pipeline documentation. The architecture is designed to minimize BatchData API costs while maintaining classification accuracy, by applying aggressive zero-cost local filtering before any paid API call is made.

Pipeline Summary

Table 1 — Pipeline Architecture Summary

Step	Operation	Outcome
Step 1	GET /properties (saleDateRange=180)	Confirmed Sales DB
Step 2a	Hard Signal Check → Phase 1 match	SOLD (no BatchData)
Step 2b	Lag Window Rule → >90 days old, no match	EXPIRED (no BatchData)

Step 2c	Delta Validation rightarrow	SKIP
Step 2d	Heuristic Scoring rightarrow Score geq3 rightarrow SOLD Score leq-4 rightarrow EXPIRED	BatchData
Step 3	Score -1 to 0 BatchData (AMBIGUOUS only)	Final classification

Expected cost reduction: \$172/mo rightarrow ~\$40/mo per rep (76% reduction). BatchData receives only ~10% of total Inactive records — the genuinely ambiguous cases.

Step 1 — /properties Ingestion with saleDateRange=180

Endpoint

GET <https://api.rentcast.io/v1/properties>

Query Parameters

Table 2 — Step 1 Query Parameters

Parameter	Value
zipCode	{ZIP}
saleDateRange	180
limit	500
offset	{offset}

Headers

Table 3 — Required HTTP Headers

Header	Value
X-API-Key	{API_KEY}
Accept	application/json

Pagination Logic

```
let offset = 0;let hasMore = true;while (hasMore) {const response = await fetchProperties({
zipCode, saleDateRange: 180, limit: 500, offset });if (!response || response.length === 0) {hasMore
```

```
= false;break;}await processAndUpsertBatch(response);if (response.length < 500) {hasMore = false;
} else {offset += 500;}}
```

Deduplication / Upsert Logic

- Primary key: formattedAddress (fallback to id if present)
- On conflict: update record only if incoming lastSaleDate is newer than stored value
- Never overwrite a newer record with an older one

Data Quality Validation (run on every record before insert)

Table 4 — Data Quality Validation Rules

Field	Validation Rule	Action on Failure
lastSaleDate	Must be ISO 8601 date within last 180 days	DISCARD record
lastSaleDate	Must not be null	DISCARD record
lastSalePrice	Must be > \$100	DISCARD (non-arm's length: \$0, \$10, \$100 are trust/LLC transfers)
owner.names	Check for: LLC, Trust, Bank, Inc, Holdings, Properties (case-insensitive)	ROUTE to investor workflow
propertyType	Must be: Single Family, Condominium, Townhouse, or Multi-Family	DISCARD Commercial/Industrial/Vacant Land
ownerOccupied	If null	FLAG for manual verification, do not discard

Weekly Refresh Schedule

- Run every Monday
- Always use saleDateRange=180 — do not reduce the window
- Rationale: Deeds recorded this week may reflect closings from 6 weeks ago. Reducing the window causes missed records on every run.

Step 2 — Zero-Cost Local Classification

CRITICAL CONSTRAINT: Do NOT reference statusCategory anywhere in Step 2. This field does not exist in the RentCast API. Do NOT reference soldPrice anywhere in Step 2. This field does not exist on any RentCast listing endpoint.

Input

Fetch all Inactive listings:

GET <https://api.rentcast.io/v1/listings/sale?zipCode={ZIP}&status=Inactive&limit=500>

Paginate using the same offset pattern as Step 1.

Step 2a — Hard Signal Check

For each Inactive listing, query the confirmed sales database (populated by Step 1):

```
const phase1Match = await db.query('SELECT lastSaleDate FROM confirmed_sales WHERE formattedAddress = $1',[listing.formattedAddress]);if (phase1Match) {const daysDiff = Math.abs(daysBetween(phase1Match.lastSaleDate, listing.removedDate));if (daysDiff <= 45) {// Strong match: deed date and removal date align within 45 daysawait classify(listing.listingId, 'SOLD', 'phase1_confirmed', 1.0);continue; // Skip BatchData}}// No match or date mismatch rightarrow proceed to Step 2b
```

Optimization note: The `saleDateRange=0:45` syntax (available since August 2025) can be used to target only very recent sales when doing targeted address lookups.

Step 2b — Lag Window Rule

```
const daysSinceRemoval = daysBetween(listing.removedDate, today);if (daysSinceRemoval > 90) {const dbMatch = await db.query('SELECT id FROM confirmed_sales WHERE formattedAddress = $1',[listing.formattedAddress]);if (!dbMatch) {// Removed >90 days ago, no deed recorded rightarrow definitively not a recent saleawait classify(listing.listingId, 'EXPIRED_WITHDRAWN', 'lag_window_rule', 0.90);continue; // Skip BatchData}}// removedDate within last 90 days rightarrow proceed to Step 2c
```

Step 2c — Delta Validation

```
const previouslySeen = await db.query('SELECT listingId FROM listing_seen_log WHERE listingId = $1',[listing.listingId]);if (previouslySeen) {// Already processed in a prior cycle — skip entirelyawait db.query('UPDATE listing_seen_log SET lastProcessedDate = NOW() WHERE listingId = $1',[listing.listingId]);continue;}// New listing — log it and proceed to scoringawait db.query('INSERT INTO listing_seen_log (listingId, firstSeenDate, lastProcessedDate) VALUES ($1, NOW(), NOW())',[listing.listingId]);
```

Persistent Store Schema

```
CREATE TABLE listing_seen_log (listingId VARCHAR PRIMARY KEY,firstSeenDate TIMESTAMP NOT NULL,lastProcessedDate TIMESTAMP NOT NULL,classification VARCHAR,confidence FLOAT);
```

Expected throughput reduction: ~215 records/week rightarrow ~13 new records/week (~94%)

reduction after first cycle)

Step 2d — Heuristic Scoring Model

Applied only to new records that passed Steps 2a, 2b, and 2c.

Positive Signals (Sold Indicators)

Table 5 — Positive Scoring Signals

Condition	Points	Rationale
daysOnMarket < 30	+3	Fast sale = strong sold signal
(removedDate - listedDate) < 45 days	+2	Short listing duration = likely sold
(removedDate - lastSeenDate) >= 7 days	+1	Gradual removal = sold pattern (not abrupt withdrawal)
history object has single entry only	+1	No relisting history = clean sale

Negative Signals (Expired/Withdrawn Indicators)

Table 6 — Negative Scoring Signals

Condition	Points	Rationale
daysOnMarket > 150	-3	Long market time = likely expired
daysOnMarket approx 90, 180, or 365 (plusminus3 days)	AUTO-REJECT as EXPIRED	Contract expiration boundary — these are almost always expirations
(removedDate - listedDate) approx 90, 180, or 365 days (plusminus3 days)	AUTO-REJECT as EXPIRED	Listing duration matches standard contract terms
(removedDate - lastSeenDate) = 0 days	-2	Same-day removal = abrupt withdrawal pattern
(removedDate - listedDate) < 7 days	-1	Very short listing = likely withdrawn before market
history object has 3+ prior entries	-1	Repeated relisting = distressed/difficult property

Classification Thresholds

Table 7 — Heuristic Classification Decision Thresholds

Score	Classification	BatchData?	Confidence
geq 3	LIKELY_SOLD	No	75–85%
1–2	PROBABLY_SOLD	No	60–70%
-1 to 0	AMBIGUOUS	Yes → Step 3	N/A
-2 to -3	PROBABLY_EXPIRED	No	60–70%
leq -4	LIKELY_EXPIRED_WITHDRAWN	No	75–85%

Scoring Implementation

```
function scoreInactiveListing(listing: InactiveListing): ScoringResult {
  let score = 0;
  const signals: string[] = [];
  // AUTO-REJECT checks first
  const dom = listing.daysOnMarket;
  const listingDuration = daysBetween(listing.listedDate, listing.removedDate);
  const contractBoundaries = [90, 180, 365];
  for (const boundary of contractBoundaries) {
    if (Math.abs(dom - boundary) <= 3) {
      return { classification: 'LIKELY_EXPIRED_WITHDRAWN', score: -99, signals: ['auto_reject_dom_${boundary}'], confidence: 0.92 };
    }
    if (Math.abs(listingDuration - boundary) <= 3) {
      return { classification: 'LIKELY_EXPIRED_WITHDRAWN', score: -99, signals: ['auto_reject_duration_${boundary}'], confidence: 0.92 };
    }
  }
  // Positive signals
  if (dom < 30) { score += 3; signals.push('fast_sale'); }
  if (listingDuration < 45) { score += 2; signals.push('short_duration'); }
  const daysSinceLastSeen = daysBetween(listing.lastSeenDate, listing.removedDate);
  if (daysSinceLastSeen >= 7) { score += 1; signals.push('gradual_removal'); }
  const historyCount = listing.history?.length ?? 0;
  if (historyCount === 1) { score += 1; signals.push('clean_history'); }
  // Negative signals
  if (dom > 150) { score -= 3; signals.push('long_dom'); }
  if (daysSinceLastSeen === 0) { score -= 2; signals.push('abrupt_removal'); }
  if (listingDuration < 7) { score -= 1; signals.push('very_short_listing'); }
  if (historyCount >= 3) { score -= 1; signals.push('repeated_relisting'); }
  // Classify
  let classification: string;
  let confidence: number;
  if (score >= 3) { classification = 'LIKELY_SOLD'; confidence = 0.80; }
  else if (score >= 1) { classification = 'PROBABLY_SOLD'; confidence = 0.65; }
  else if (score >= -1) { classification = 'AMBIGUOUS'; confidence = 0.0; }
  else if (score >= -3) { classification = 'PROBABLY_EXPIRED'; confidence = 0.65; }
  else { classification = 'LIKELY_EXPIRED_WITHDRAWN'; confidence = 0.80; }
  return { classification, score, signals, confidence };
}
```

Step 3 — BatchData (Last Resort Only)

Input

Only AMBIGUOUS records (score -1 to 0) from Step 2d. Estimated ~10% of total Inactive pool (~21 records/area).

POST https://api.batchdata.com/api/v1/property/lookup/async

Request Payload

```
{"properties": [{"address": "123 Main St", "city": "Austin", "state": "TX", "zip": "78701"}], "options": {"webhookUrl": "{BASE44_WEBHOOK_URL}/batchDataWebhookCallback", "requestId": "{jobId}"}}
```

Webhook Callback Handler (batchDataWebhookCallback)

NOTE: statusCategory and listing.status ARE valid BatchData fields. These are BatchData's own fields — NOT RentCast fields. It is correct and expected to reference them here in the BatchData webhook handler.

```
async function batchDataWebhookCallback(payload: BatchDataWebhookPayload) {const {
requestId, data } = payload;for (const property of data.results.properties) {// NOTE: statusCategory
and listing.status ARE valid BatchData fields// These are BatchData's own fields — NOT RentCast
fieldsconst { statusCategory, status } = property.listing;let classification: string;if (statusCategory
=== 'Sold' || status === 'Sold') {classification = 'SOLD_CONFIRMED';} else if (statusCategory ===
'Pending') {classification = 'PENDING_CONFIRMATION';} else {classification =
'EXPIRED_WITHDRAWN_CONFIRMED';}// Upsert to PropertyValidationCacheawait db.query(`
INSERT INTO property_validation_cache (formattedAddress, classification, source, updatedAt)
VALUES ($1, $2, 'batchdata', NOW())ON CONFLICT (formattedAddress) DO UPDATESET
classification = $2, source = 'batchdata', updatedAt = NOW()`, [property.formattedAddress,
classification]);// Update ValidationQueueawait db.query(`UPDATE validation_queue SET status =
'completed', classification = $1, completedAt = NOW()WHERE requestId = $2 AND
formattedAddress = $3`, [classification, requestId, property.formattedAddress]);}}
```

Cost Projection

Table 8 — BatchData Cost Projection by Scenario

Scenario	Records/Area	Cost/Area	Areas/Mo	Monthly Cost/Rep
Current (broken)	~215	~\$21.50	8	~\$172
Optimized (this spec)	~21	~\$2.10	8	~\$16.80
Growth tier breakeven	—	—	—	25+ reps

Redfin Pending Cross-Check (Optional Enhancement)

For properties classified as PROBABLY_SOLD or AMBIGUOUS, implement an optional Redfin

cross-check:

```
async function redfinCrossCheck(formattedAddress: string): Promise<'pending' | 'active' | 'sold' | 'unknown'> {  
  // Query Redfin search API or scrape listing page// If Redfin shows 'Pending'  
  rightarrow hold in pending_confirmation queue// Re-check in 30 days}
```

Decision Rule

If RentCast shows Inactive AND Redfin shows Pending rightarrow classify as PENDING_CONFIRMATION, hold in queue, re-check in 30 days. Do not send to door-knocking queue until confirmed.

Rollout Plan

Week 1–2: Shadow Mode

- Deploy new heuristic scoring alongside existing full BatchData flow
- Log heuristic classifications but do not act on them
- Compare heuristic output vs BatchData output for every record
- Target: heuristic accuracy geq 80% vs BatchData ground truth

Week 3–4: Partial Cutover

- Trust Phase 1 matches (Step 2a) rightarrow skip BatchData
- Trust extreme scores (geq 4 or leq -3) rightarrow skip BatchData
- Send middle scores (-1 to 3) to BatchData
- Expected cost reduction: ~50%

Month 2+: Full Automation

- Full heuristic pipeline live
- BatchData receives only AMBIGUOUS records (score -1 to 0)
- Monitoring dashboard tracks:
 - heuristic_accuracy (validated against BatchData spot-checks)
 - batchdata_calls_per_run
 - cost_per_area
 - false_positive_rate (sold leads that turn out to be expired)
 - false_negative_rate (expired leads that turn out to be sold)

Monitoring Alerts

Table 9 — Monitoring Alert Thresholds

Alert Condition	Threshold	Meaning
batchdata_calls_per_run > 30	> 30 calls	Upstream filter regression — investigate Step 2 filters
false_positive_rate > 15%	> 15%	Heuristic model degradation — retune scoring weights
cost_per_area > \$5	> \$5.00	Cost regression — check for new record volume spikes

Database Schema Changes Required

listing_seen_log — Delta Validation Log

CREATE TABLE listing_seen_log (listingId VARCHAR PRIMARY KEY,firstSeenDate TIMESTAMP NOT NULL,lastProcessedDate TIMESTAMP NOT NULL,classification VARCHAR,confidence FLOAT);

heuristic_score_log — Heuristic Scoring Audit Log

CREATE TABLE heuristic_score_log (id SERIAL PRIMARY KEY,listingId VARCHAR NOT NULL,score INTEGER NOT NULL,signals JSONB,classification VARCHAR NOT NULL,confidence FLOAT,batchdataResult VARCHAR, -- populated during shadow modecreatedAt TIMESTAMP DEFAULTNOW());

pending_confirmation_queue — Pending Confirmation Queue

CREATE TABLE pending_confirmation_queue (listingId VARCHAR PRIMARY KEY,formattedAddress VARCHAR NOT NULL,classifiedAt TIMESTAMP NOT NULL,reCheckDate TIMESTAMP NOT NULL, -- classifiedAt + 30 daysstatus VARCHAR DEFAULT'pending');

Schema Summary

Table 10 — New Database Tables Summary

Table Name	Purpose	Primary Key	Key Fields
listing_seen_log	Delta validation — tracks all previously seen listing IDs to prevent reprocessing	listingId	firstSeenDate, lastProcessedDate, classification, confidence

heuristic_score_log	Audit trail for all heuristic scoring decisions; used for shadow mode	id (serial)	listingId, score, signals (JSONB), classification, batchdataResult
pending_confirmation_queue	Holds PROBABLY_SOLD and AMBIGUOUS records pending Redfin cross-check or re-evaluation	listingId	formattedAddress, classifiedAt, reCheckDate, status

End of FirstKnock Revised Pipeline Architecture Spec